

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Code Coverage Report

Blood Bank Management System

Version 1.0

Prepared by Team 9 — Purav N Khincha (PES1UG24CS349)

Testing & QA Lead

F-Section, PES University

05/11/2026

Tested commit: 61d576d (main)

Revision History

Version	Date	Details
1.0	05/11/2026	Initial statement coverage report for BBMS at commit 61d576d.

Summary

Item	Result
Tested commit	61d576d — Merge pull request #50 (E5-S3 reports and history), 01/11/2026
Test run	131 tests run — 129 passed, 0 failed, 2 skipped (exit code 0)
Overall statement coverage	89.47% (756 of 845 statements)
Apps meeting the 80% target	8 of 9
App below target	issues — 69.96%, caused entirely by the performance-data seeding command seed_perf_data.py (0 of 57 statements run). Without that tooling file: 93.98%.

1. Objective and Scope

1.1 Objective

This report measures how much of the BBMS application code is executed by the automated Django test suite, using statement (line) coverage from coverage.py. It checks each Django app against the Software Test Plan target of **at least 80% statement coverage per app** (STP v1.0, Section 5 Test Levels and Section 14 Test Metrics), and lists the lines that are not executed together with proposed tests.

1.2 Scope

Included: all nine BBMS application packages — accounts, donors, donations, inventory, monitoring, blood_search, blood_requests, issues and reports.

Excluded: database migrations (`*/migrations/*`), test files (`*/tests.py`, `*/test_*.py`), the Django project settings package (config), the virtual environment and third-party libraries. Branch (decision) coverage is reported separately in the Branch Coverage Report.

1.3 Reference Documents

- BBMS Software Requirements Specification (SRS), Version 1.0.
- BBMS Software Architecture and Design Specification (SAD), Version 1.0.
- BBMS Software Test Plan (STP), Version 1.0 — Sections 5 and 14 (coverage target and metrics).
- BBMS Unit Test Report, Version 1.0 (same commit 61d576d).
- BBMS source code, Git revision 61d576d4caa3d6fac55a9ccd4205509ae8bfe1cd.

2. Test Environment

Parameter	Details
Operating system	Linux (x86-64), command-line run
Python	3.13.16 (virtual environment)
Django	5.2.18
Coverage tool	coverage.py 7.16.2 (C extension)
Test runner	Django test runner (manage.py test), Django TestCase
Database	SQLite test database created and destroyed by Django for the run (USE_SQLITE=True)
Source revision	61d576d4caa3d6fac55a9ccd4205509ae8bfe1cd (GitHub main)
Run date	05/11/2026
Settings	DEBUG=False, ALLOWED_HOSTS=127.0.0.1,localhost,testserver, local test secret key

No real donor or patient data was used. Django created a temporary test database, applied all migrations and removed it after the run.

3. Test Method

3.1 Commands

The full automated suite was run under coverage.py with the scope given in Section 1.2, then text, JSON and HTML reports were generated:

```
python -m coverage erase
python -m coverage run --source=accounts,donors,donations,inventory,monitoring,
    blood_search,blood_requests,issues,reports
    --omit="*/migrations/*,*/tests.py,*/test_*.py" manage.py test
python -m coverage report -m
python -m coverage json -o Docs/Testing/Evidence/CodeCoverage/statement_coverage.json
python -m coverage html -d htmlcov
```

Branch tracking was **not** enabled in this run, so every percentage below is pure statement coverage (no combined statement/branch score).

3.2 Calculation

Statement coverage = covered statements ÷ total executable statements × 100.

Per-app figures were calculated by summing `covered_lines` and `num_statements` across every measured file of that app in the coverage JSON, then dividing. Empty `__init__.py` files contain 0 statements and therefore do not affect the result. A file-level percentage is never used as an app percentage.

3.3 Pass criterion

An app **meets** the target when its statement coverage is ≥ 80%, as defined in STP Sections 5 and 14.

4. Results

4.1 Test run

The coverage-instrumented run executed **131 tests in 103.0 s — OK (skipped=2)**. No test failed. The two skipped tests are the PostgreSQL-only concurrent-issue test and the browser portability test (needs BROWSER_TESTS and Playwright); this matches the Unit Test Report for the same commit.

4.2 Statement coverage by Django app

Django app	Covered	Total	Missed	Statement %	vs. 80% target
accounts	48	50	2	96.00%	Meets
donors	96	98	2	97.96%	Meets
donations	75	85	10	88.24%	Meets
inventory	49	51	2	96.08%	Meets
monitoring	6	7	1	85.71%	Meets
blood_search	38	39	1	97.44%	Meets
blood_requests	187	191	4	97.91%	Meets
issues	156	223	67	69.96%	Below target
reports	101	101	0	100.00%	Meets
Total (all 9 apps)	**756**	**845**	**89**	**89.47%**	**—**

Overall: 756 of 845 statements executed — **89.47%. 8 of 9 apps meet the 80% target.** The issues app is below target only because of one tooling file (Section 5.1).

4.3 Files with missed statements

All other measured application files reached 100% statement coverage. The table lists every file with at least one missed statement, the exact line numbers and what that code does.

File	Stmt %	Missing lines	What the missed code is
accounts/models.py	91%	29	UserProfile.__str__ (display text)
accounts/views.py	0%	All (1)	Default placeholder import; app has no view code
blood_requests/models.py	97%	112	BloodRequest.__str__ (display text)
blood_requests/ permissions.py	88%	23, 26, 62	user_has_role(): anonymous and superuser shortcuts; can_review_request(): anonymous shortcut
blood_search/views.py	96%	24	Superuser shortcut in can_search_availability()
donations/forms.py	93%	39	Error when expiry date is before donation date
donations/models.py	93%	25, 72	Future donation-date validator; Donation.__str__
donations/views.py	77%	27, 53, 55–56, 58, 60–61	403 for non-staff; loop that turns form errors into messages
donors/models.py	97%	78	Donor.__str__ (display text)
donors/views.py	98%	95	403 when non-staff opens donor history
inventory/services.py	96%	29	Invalid blood group returns 0 units
inventory/views.py	92%	26	403 for users with no BBMS role
issues/admin.py	78%	12, 15, 46, 49, 52	Read-only admin: block add/change/delete of issue records
issues/issuance.py	97%	108	Defensive break when the remaining need is already 0
issues/management/ commands/ seed_perf_data.py	0%	All 57 (1–182)	Whole file — performance-test data seeding command, never run by tests
issues/models.py	93%	67, 95	BloodIssue / allocation __str__ (display text)
issues/views.py	96%	71–72	Malformed JSON body in POST /api/issues
monitoring/views.py	0%	All (1)	Default placeholder import; app has no view code

5. Findings and Gaps

5.1 App below target: issues (69.96%)

`issues/management/commands/seed_perf_data.py` contributes 57 of the 67 missed statements in the issues app. It is a developer tool that generates bulk synthetic data (`performance/perf_config.json`) for the Locust load test (TC-PERF-01, 20-user part); it is not part of any user-facing workflow and no automated test calls it.

Excluding this file, the issues app's application code is at **93.98%** (156 of 166), and the overall figure would be 95.94%. Both numbers are given for transparency; the official measured value in this report remains **69.96%**, because the file is inside the agreed measurement scope.

Recommended action (either):

- Add one smoke test that runs `call_command("seed_perf_data", ...)` with a very small data size and checks the created records and config file, or
- Agree as a team to classify management commands used only for load-test setup as tooling and add them to the coverage `omit` list, recording the decision in the Test Plan.

5.2 Untested negative paths

Gap	Where	Proposed additional test
Non-staff user records a donation (403)	donations/views.py:27	POST /api/donations as Hospital Staff → expect 403 and no Donation created (TC-SEC-02)
Invalid donation submitted through the view shows error messages	donations/views.py:53–61	POST with units = 0 through the view → expect error message and no record (TC-DNT-02)
Expiry date earlier than donation date	donations/forms.py:39	Form with expiry < donation date → expect error on expiry_date
Donation date in the future (model validator)	donations/models.py:25	Call full_clean() on a Donation dated tomorrow → expect ValidationError
Non-staff user opens donor history (403)	donors/views.py:95	GET /api/donors/<id> as Hospital Staff → expect 403
User without a BBMS role opens inventory (403)	inventory/views.py:26	GET /inventory/ as a user with no group → expect 403
Invalid blood group passed to availability service	inventory/services.py:29	get_available_units("XYZ") → expect 0
Malformed JSON body sent to issue API	issues/views.py:71–72	POST /api/issues with body "{bad" → expect 400-style error, no stock change
Superuser / anonymous shortcuts in role helpers	blood_requests/ permissions.py:23, 26, 62; blood_search/views.py:24	Direct unit tests of user_has_role(), can_review_request() and can_search_availability() for anonymous and superuser users

5.3 Low-risk uncovered code

- **`__str__` methods** — accounts/models.py:29, blood_requests/models.py:112, donations/models.py:72, donors/models.py:78, issues/models.py:67, 95. Only used for display in the admin; one assertEqual(str(obj), ...) per model would cover them.
- **Read-only admin permissions** — issues/admin.py:12, 15, 46, 49, 52. These block add/change/delete of issue records in the Django admin. A small admin test as a superuser would confirm the audit trail cannot be edited.
- **Defensive `break`** — issues/issuance.py:108. Reached only when the allocation plan contains more batches than needed; a test with three small batches for a two-unit issue would execute it.
- **Empty placeholder views** — accounts/views.py and monitoring/views.py contain only the default Django import line (no view code). Monitoring is implemented in the inventory app's services and dashboard.

5.4 Limitations

- Coverage was measured on the SQLite test database. The PostgreSQL-only concurrent-issue test was skipped, so any code that runs only under PostgreSQL row locking is not reflected.
- The browser portability test was skipped in this run; the views it drives are already covered by Django test-client tests.
- Statement coverage shows which lines ran, not that every decision outcome was checked; see the Branch Coverage Report.

5.5 STP exit criterion — code coverage

Criterion (STP Section 14)	Status	Evidence
Statement coverage ≥ 80% per Django app	Partially met — 8 of 9 apps	Section 4.2; statement_coverage.json
issues app ≥ 80%	Not met (69.96%) — 93.98% excluding tooling file	Section 5.1

6. Conclusion

At commit 61d576d, the automated suite (131 tests, 0 failures, 2 environment-related skips) executes **89.47% of BBMS application statements**. Eight of the nine apps meet the Test Plan's 80% statement coverage target, with reports, blood_requests, donors, blood_search, accounts and inventory all above 96%.

The issues app is measured at 69.96%, below target, because its performance-data seeding command is never run by the tests; its user-facing issue workflow code is above 93%. Adding one smoke test for that command, plus the negative-path tests in Section 5.2, would bring every app above 80% and close the remaining permission-denied and invalid-input gaps.

Appendix A. Coverage Terminal Report

Output of `python -m coverage report -m` for the run described in this report (saved as Evidence/CodeCoverage/statement_summary.txt).

```
Name                                                 Stmts   Miss  Cover   Missing
----------------------------------------------------------------------------------
accounts/__init__.py                                     0      0   100%
accounts/admin.py                                        6      0   100%
accounts/apps.py                                         6      0   100%
accounts/decorators.py                                  14      0   100%
accounts/models.py                                      11      1    91%   29
accounts/signals.py                                      8      0   100%
accounts/urls.py                                         4      0   100%
accounts/views.py                                        1      1     0%   1
blood_requests/__init__.py                               0      0   100%
blood_requests/admin.py                                  8      0   100%
blood_requests/apps.py                                   4      0   100%
blood_requests/forms.py                                 24      0   100%
blood_requests/models.py                                33      1    97%   112
blood_requests/permissions.py                           24      3    88%   23, 26, 62
blood_requests/services.py                              12      0   100%
blood_requests/urls.py                                   4      0   100%
blood_requests/views.py                                 82      0   100%
blood_search/__init__.py                                 0      0   100%
blood_search/admin.py                                    1      0   100%
blood_search/apps.py                                     4      0   100%
blood_search/models.py                                   1      0   100%
blood_search/services.py                                 6      0   100%
blood_search/urls.py                                     4      0   100%
blood_search/views.py                                   23      1    96%   24
donations/__init__.py                                    0      0   100%
donations/admin.py                                       1      0   100%
donations/apps.py                                        4      0   100%
donations/forms.py                                      15      1    93%   39
donations/models.py                                     30      2    93%   25, 72
donations/urls.py                                        4      0   100%
donations/views.py                                      31      7    77%   27, 53-61
donors/__init__.py                                       0      0   100%
donors/admin.py                                          1      0   100%
donors/apps.py                                           4      0   100%
donors/forms.py                                         10      0   100%
donors/models.py                                        30      1    97%   78
donors/urls.py                                           4      0   100%
donors/views.py                                         49      1    98%   95
inventory/__init__.py                                    0      0   100%
inventory/admin.py                                       1      0   100%
inventory/apps.py                                        4      0   100%
inventory/models.py                                      1      0   100%
inventory/services.py                                   28      1    96%   29
inventory/urls.py                                        4      0   100%
inventory/views.py                                      13      1    92%   26
issues/__init__.py                                       0      0   100%
issues/admin.py                                         23      5    78%   12, 15, 46, 49,
52
issues/apps.py                                           4      0   100%
issues/issuance.py                                      39      1    97%   108
issues/management/__init__.py                            0      0   100%
issues/management/commands/__init__.py                   0      0   100%
issues/management/commands/seed_perf_data.py            57     57     0%   1-182
issues/models.py                                        28      2    93%   67, 95
issues/permissions.py                                    3      0   100%
issues/services.py                                       9      0   100%
issues/urls.py                                           4      0   100%
issues/views.py                                         56      2    96%   71-72
monitoring/__init__.py                                   0      0   100%
monitoring/admin.py                                      1      0   100%
monitoring/apps.py                                       4      0   100%
monitoring/models.py                                     1      0   100%
monitoring/views.py                                      1      1     0%   1
```

```
reports/__init__.py                                 0      0   100%
reports/admin.py                                    1      0   100%
reports/apps.py                                     4      0   100%
reports/forms.py                                   21      0   100%
reports/models.py                                   1      0   100%
reports/permissions.py                              3      0   100%
reports/services.py                                39      0   100%
reports/urls.py                                     4      0   100%
reports/views.py                                   28      0   100%
-------------------------------------------------------------------------
TOTAL                                             845     89    89%
```

Appendix B. HTML Coverage Report Screenshots

BBMS statement coverage - 61d576d: 89%

Files | Functions | Classes

filter... hide covered

coverage.py v7.16.2, created at 2026-11-05 02:56 +0000

File ▲	statements	missing	excluded	coverage
accounts/__init__.py	0	0	0	100%
accounts/admin.py	6	0	0	100%
accounts/apps.py	6	0	0	100%
accounts/decorators.py	14	0	0	100%
accounts/models.py	11	1	0	91%
accounts/signals.py	8	0	0	100%
accounts/urls.py	4	0	0	100%
accounts/views.py	1	1	0	0%
blood_requests/__init__.py	0	0	0	100%
blood_requests/admin.py	8	0	0	100%
blood_requests/apps.py	4	0	0	100%
blood_requests/forms.py	24	0	0	100%
blood_requests/models.py	33	1	0	97%
blood_requests/permissions.py	24	3	0	88%
blood_requests/services.py	12	0	0	100%
blood_requests/urls.py	4	0	0	100%
blood_requests/views.py	82	0	0	100%
blood_search/__init__.py	0	0	0	100%
blood_search/admin.py	1	0	0	100%
blood_search/apps.py	4	0	0	100%
blood_search/models.py	1	0	0	100%
blood_search/services.py	6	0	0	100%
blood_search/urls.py	4	0	0	100%
blood_search/views.py	23	1	0	96%
donations/__init__.py	0	0	0	100%
donations/admin.py	1	0	0	100%
donations/apps.py	4	0	0	100%
donations/forms.py	15	1	0	93%
donations/models.py	30	2	0	93%
donations/urls.py	4	0	0	100%
donations/views.py	31	7	0	77%
donors/__init__.py	0	0	0	100%
donors/admin.py	1	0	0	100%
donors/apps.py	4	0	0	100%
donors/forms.py	10	0	0	100%
donors/models.py	30	1	0	97%

Figure B1. coverage.py HTML index — files A to I

donors/forms.py	10	0	0	100%
donors/models.py	30	1	0	97%
donors/urls.py	4	0	0	100%
donors/views.py	49	1	0	98%
inventory/__init__.py	0	0	0	100%
inventory/admin.py	1	0	0	100%
inventory/apps.py	4	0	0	100%
inventory/models.py	1	0	0	100%
inventory/services.py	28	1	0	96%
inventory/urls.py	4	0	0	100%
inventory/views.py	13	1	0	92%
issues/__init__.py	0	0	0	100%
issues/admin.py	23	5	0	78%
issues/apps.py	4	0	0	100%
issues/issuance.py	39	1	0	97%
issues/management/__init__.py	0	0	0	100%
issues/management/commands/__init__.py	0	0	0	100%
issues/management/commands/seed_perf_data.py	57	57	0	0%
issues/models.py	28	2	0	93%
issues/permissions.py	3	0	0	100%
issues/services.py	9	0	0	100%
issues/urls.py	4	0	0	100%
issues/views.py	56	2	0	96%
monitoring/__init__.py	0	0	0	100%
monitoring/admin.py	1	0	0	100%
monitoring/apps.py	4	0	0	100%
monitoring/models.py	1	0	0	100%
monitoring/views.py	1	1	0	0%
reports/__init__.py	0	0	0	100%
reports/admin.py	1	0	0	100%
reports/apps.py	4	0	0	100%
reports/forms.py	21	0	0	100%
reports/models.py	1	0	0	100%
reports/permissions.py	3	0	0	100%
reports/services.py	39	0	0	100%
reports/urls.py	4	0	0	100%
reports/views.py	28	0	0	100%
Total	**845**	**89**	**0**	**89%**

coverage.py v7.16.2, created at 2026-11-05 02:56 +0000

Figure B2. coverage.py HTML index — files I to Z and total

Coverage for **donations/views.py**: 77%

31 statements | 24 run | 7 missing | 0 excluded

« prev ^ index » next coverage.py v7.16.2, created at 2026-11-05 02:56 +0000

```
1  from urllib.parse import urlencode
2
3  from django.contrib import messages
4  from django.contrib.auth.decorators import login_required
5  from django.core.exceptions import PermissionDenied
6  from django.shortcuts import get_object_or_404, redirect
7  from django.urls import reverse
8  from django.views.decorators.http import require_POST
9
10 from donors.models import Donor
11
12 from .forms import DonationRecordingForm
13
14
15 @login_required
16 @require_POST
17 def record_donation(request):
18     """
19     Record a blood donation for a registered donor.
20
21     Only authenticated Blood Bank Staff may record donations.
22     """
23
24     if not request.user.groups.filter(
25         name="Blood Bank Staff"
26     ).exists():
27         raise PermissionDenied(
28             "You are not authorized to record donations."
29         )
30
31     donor_id = request.POST.get("donor_id", "").strip()
32
33     donor = get_object_or_404(
34         Donor,
35         donor_id=donor_id,
36     )
37
38     form = DonationRecordingForm(request.POST)
39
40     if form.is_valid():
41         donation = form.save(commit=False)
42
43         donation.donor = donor
44         donation.save()
45
46         messages.success(
47             request,
48             f"Donation {donation.donation_id} recorded successfully "
49             f"for {donor.full_name}.",
50         )
51
52     else:
53         for field_name, errors in form.errors.items():
54
55             if field_name in form.fields:
56                 label = form.fields[field_name].label
57             else:
58                 label = "Donation"
59
60             for error in errors:
61                 messages.error(
62                     request,
63                     f"{label}: {error}",
64                 )
65
66     search_url = reverse("donors:search")
```

Figure B3. donations/views.py — missed lines highlighted in red (permission denied and form-error branch)

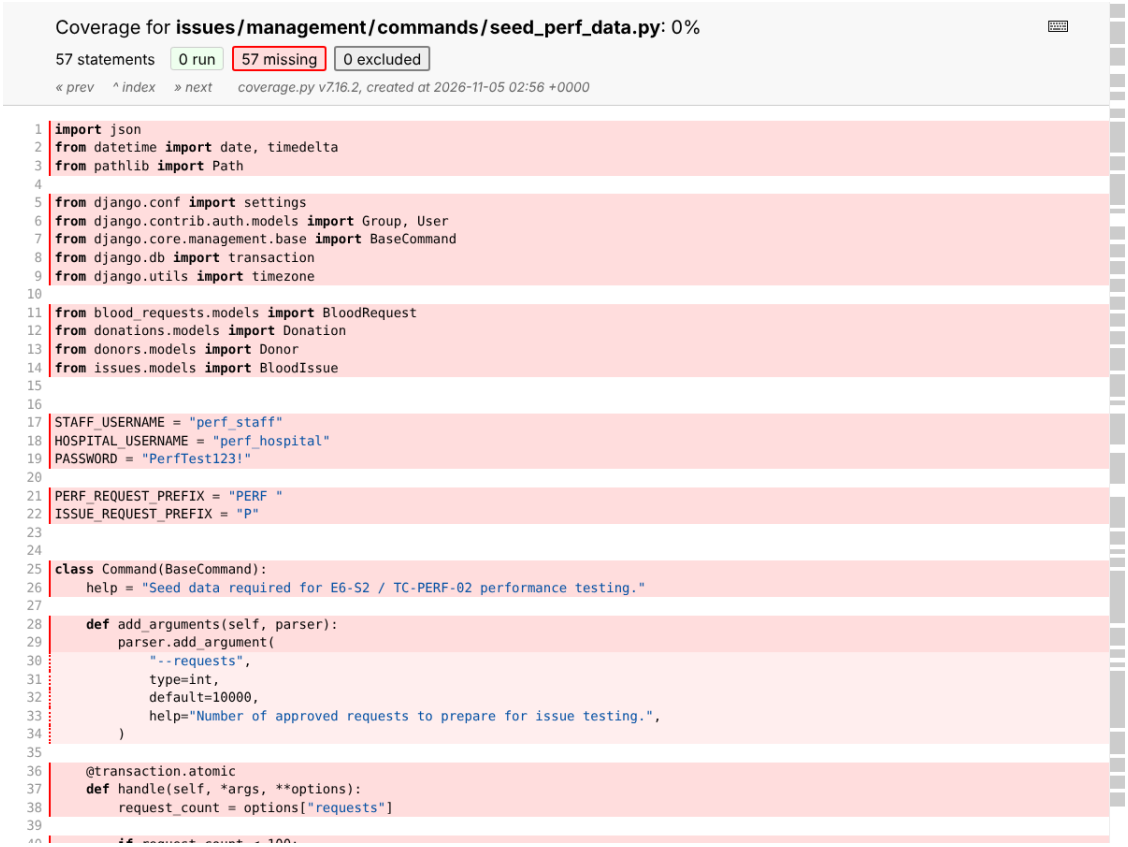


Figure B4. issues/management/commands/seed_perf_data.py — not executed by the test suite

Appendix C. Evidence Files

File (Docs/Testing/Evidence/CodeCoverage/)	Contents
statement_summary.txt	Terminal coverage report with missing line numbers
statement_coverage.json	Machine-readable coverage data (per-file covered/missing lines and totals)
B1_coverage_index.png	Screenshot of the HTML report index
B2_donations_views.png	Screenshot of donations/views.py coverage view
B3_seed_perf_data.png	Screenshot of seed_perf_data.py coverage view

The full HTML report (htmlcov/) is not committed because it is generated output and is listed in .gitignore; it can be regenerated with the commands in Section 3.1.